

Audion Media Tools - User Guide

Актуально для **Audion Media Tools** на 2026-06-15.

Это пользовательский guide по всем основным возможностям проекта: от выбора **Source** / **OUT** до yt-dlp, аудиопотоков, архивных и монтажных кодеков, публикации, remux, FPS, LUT/grading, диагностики, отчётов и ручного терминала.

Если нужен быстрый технический обзор, смотри **README.md** и **Docs\README_RU.md**. Если нужен глубокий разбор по разделам, смотри файлы в **Docs\docs**, перечисленные ниже.

Что Это

Audion Media Tools - портативная Windows-оболочка для реальной работы с медиа. Внутри проекта лежат runtime, FFmpeg/ffprobe, yt-dlp, Deno, 7-Zip, NiceGUI shell, профили операций и старый CLI-слой **Scripts**.

Проект не является абстрактным FFmpeg-frontend. Его смысл в другом: собрать проверенные media-процедуры в один управляемый контур с профилями, dry-run, отчётами, логами, path-cache, LUT-cache, hardware checks и понятными подсказками над каждым важным режимом.

Быстрый Старт

Обычный запуск GUI:

```
launcher_gui.cmd
```

Если portable-окружение не собрано:

```
builder_main.cmd
```

GUI открывает локальный NiceGUI shell. Значение по умолчанию:

```
http://127.0.0.1:8080/
```

Если порт занят тестовым экземпляром, можно запускать вручную на другом порту:

```
runtime\python.exe system_core\ui_nicegui\app.py --host 127.0.0.1 --port 8081 --no-browser
```

Главная Рабочая Логика

Верхний блок задаёт два глобальных пути:

- **Source** - папка входных файлов.
- **OUT** / **Transcoded** - папка назначения.

Пути выбираются кнопками рядом с полями. Dropdown-поля показывают историю и закреплённые пути. Пин поднимает путь вверх. Кнопки очистки требуют подтверждения.

Source для обычных batch media-операций обходится рекурсивно. OUT повторяет структуру подпапок Source. Результаты пишутся как **OUT\<operation>\<suffix>\<зеркало Source>\<исходное имя>.<ext>**. Если выбран кастомный OUT, он становится корнем этого же правила. Это важно: **Source\Day1\clip.mov** и **Source\Day2\clip.mov** больше не должны сваливаться в одну плоскую папку и конфликтовать по имени.

Tooltips

В интерфейсе есть подсказки над кнопками, режимами и параметрами. Задержки:

- показ: **1500 ms**;
- скрытие: **100 ms**.

Фон подсказок унифицирован как **RGB(23, 33, 43)**. Если непонятно, что делает поле, задержи курсор примерно на полторы секунды. Tooltips особенно важны для режимов аудио, stream selection, Lufs, resampler, hardware encode/decode, remux-пар, LUT/Color и операций с побочными эффектами. Расширения файлов и очевидные кнопки вроде **Назад** или **Запустить** подсказками не дублируются.

Крайние segmented-кнопки выравнивают подсказку по краю, чтобы длинный текст не обрезался слева или справа.

Как Запускать Операцию

1. Выбери **Source** и **OUT**.
2. Открой нужную страницу слева.
3. Выбери профиль.
4. Измени только нужные параметры.
5. Нажми **Команда**, если хочешь увидеть dry-run.
6. Нажми кнопку запуска справа в верхней строке операции.

Профиль задаёт готовую базу, а поля ниже позволяют точно переопределить режим.

Чекбоксы не должны включать скрытые действия, если профиль явно их не задал.

Документы По Разделам

- **Docs\docs\AUDION_MEDIA_TOOLS_RU.md** - архитектура, папки, GUI и принципы.
- **Docs\docs\WORKSPACE_REPORTS_TERMINAL_RU.md** - Source/OUT, path-cache, терминал, отчёты.
- **Docs\docs\YOUTUBE_DOWNLOAD_RU.md** - yt-dlp profiles, batch, субтитры, cookies.
- **Docs\docs\AUDIO_WORKFLOWS_RU.md** - извлечение аудио, звук в видео, конвертация аудиофайлов.
- **Docs\docs\ENCODING_PIPELINES_RU.md** - архив, монтажные кодеки, хранение, публикация.
- **Docs\docs\REMUX_FPS_LUT_RU.md** - remux, FPS/conform/varispeed, LUT и HDR-to-SDR.
- **Docs\TRIM_RU.md** - обрезка подробно: рабочий цикл, три действия, точность, отказы.
- **Docs\MPV_HOTKEYS_RU.md** - клавиши плеера: сперва монтажные, потом все 197.
- **Docs\TRIM_MATRIX_RU.md** - что с чем совместимо при резе.
- **Docs\MEASUREMENTS_RU.md** - свод измерений: что мерили и что получилось.
- **Docs\docs\HARDWARE_DIAGNOSTICS_RU.md** - диагностика, hardware cache, Profile Doctor, Test Matrix.
- **Docs\docs\MEDIA_KNOWN_PITFALLS_RU.md** - типовые ошибки и как их обходить.
- **Docs\docs\MEDIA_SMOKE_TEST_CHECKLIST_RU.md** - проверка после изменений и перед релизом.

Страницы Операций

Диагностика

Используй первой, если проект не видит tools, профили или hardware backend.

Основные команды:

- `Inventory` - считает рабочие папки.
- `Selftest` - проверяет FFmpeg, ffprobe, yt-dlp и Deno.
- `Profile Doctor` - сверяет manifest, `Scripts\*.cmd` и `config\script_profiles.yaml`.
- `Hardware Capabilities` - проверяет CPU, CUDA/NVENC, QSV, AMF/D3D11VA (H.264, HEVC и AV1), SVT-AV1 и dav1d.
- `Preset Test Matrix` - прогоняет пресеты на коротком fixture-файле и отличает `MISS` от настоящей ошибки.

Установка Инструментов

Раздел для явной установки tool payloads:

- 7-Zip;
- FFmpeg;
- yt-dlp.

Для полной сборки окружения обычно удобнее `builder_main.cmd`, но GUI-раздел полезен для точечного ремонта.

Скачивание / YouTube

Работает через yt-dlp. Поддерживает один URL или batch из `Download\urls.txt`.

Ключевые профили:

- `Best MP4 video` - лучший общий MP4/M4A merge.
- `1080p AVC MP4` - совместимый H.264/MP4.
- `Safe Safari` - Safari client, IPv4 и повторы для капризных ссылок.
- `AV1 если доступен` - предпочитает AV1.
- `Audio MP3` - audio-only MP3.
- `Batch из urls.txt` - читает список ссылок.

Ключевые параметры: разрешение, video codec, audio format, container, source mode, retry/IPv4/metadata/no-playlist, subtitles, fragments, client, languages, cookies и download archive.

Аудиопотоки

Три главных режима теперь вынесены в segmented-кнопки:

- **Извлечь аудио** - создать отдельные аудиофайлы из видео-контейнеров.
- **Звук в видео** - создать новый видеофайл: видео копируется, выбранное аудио меняется внутри контейнера.
- **Конвертировать аудио** - обработать отдельные WAV/FLAC/M4A/MP3/AAC/Opus/MKA/OGG/APE/WV и другие аудиофайлы из Source, если их читает локальный FFmpeg.

Ключевые возможности:

- Кнопки режимов: Copy/remux, lossless/PCM, lossy delivery и анализ; они меняют параметры формы.
- Copy, WAV, FLAC, ALAC, M4A/AAC, MP3, Opus.
- Copy container policy: Auto, M4A, MKA или родной файл кодека; Auto извлекает AAC stereo/mono как M4A, AC3/E-AC3/DTS как родные файлы, а сложный многоканал оставляет в MKA.
- Copy/source, 44.1/88.2/176.4 kHz и 48/96/192 kHz.
- Copy/source, 16-bit, 24-bit, 32-bit float.
- SoX/libsoxr 100% quality (**precision=33**) для изменения частоты.
- Первый поток, все потоки, поток по индексу или по языку.
- LUFs off/report/one-pass/two-pass.
- Keep, stereo, mono, dual mono, 5.1 Matrix.
- LAME V0, Insane 320 или manual bitrate. Та же шкала доступна на страницах кодирования при **Аудио = MP3**; manual bitrate ограничен 320 kbps, выше MPEG-1 Layer III не умеет.

После аудио-операций пишется **audio_report.md/json**.

Архив

Долговременные мастер-форматы:

- FFV1 MKV + FLAC;
- x264 lossless MKV;

- dual archive.

Выбирай архив, когда нужно сохранить максимальную техническую информацию и размер файла вторичен.

Монтажные Кодеки

Промежуточные форматы для NLE:

- ProRes Proxy / LT / 422 / HQ в MOV или MXF;
- DNxHR HQ / HQX в MOV или MXF.

Пресеты разделены на четыре группы: **ProRes · MOV**, **ProRes · MXF**, **DNxHR · MOV**, **DNxHR · MXF**. Кодек и контейнер остаются независимыми ручными параметрами. Для MXF используются PCM 16/24-bit или режим без аудио; несовместимые аудиорежимы блокируются до запуска.

Это правильная зона для подготовки файлов к монтажу, а не для финальной публикации.

Хранение

Компактные качественные файлы для библиотеки или передачи:

- x264 / H.264;
- x265 / HEVC;
- SVT-AV1.

Главные параметры: backend, decode backend, audio, bitrate, container, preset, resolution, AV1 preset, pixel format, CRF, tuning.

Доставка / Публикация

Профили для просмотра, отправки и заливки:

- Review H.264;
- Upload H.264;
- Upload HEVC 10-bit;
- SVT-AV1 delivery.

Аппаратное кодирование выбирается отдельно в блоке **Кодек**: CPU, NVENC, AMF или QSV. Для CPU показываются **CPU preset** и **CRF**; для hardware-кодировщиков показываются родные preset/quality-поля и **CQ/QP**.

Упаковка / Remix

Меняет контейнер или состав потоков без перекодирования, где это возможно. Source сканируется рекурсивно, а формат каждого файла определяется автоматически.

Пайплайн:

- **Video** - оставить только видеотреки.
- **Audio** - отделить аудиотреки.
- **Video&Audio** - скопировать видео и аудио вместе.

Для **Video** и **Video&Audio** выбирается целевой контейнер MP4, MOV, MKV или MXF. Для **Audio** выбирается аудиоконтейнер M4A, ALAC/M4A, MKA, AAC, AC3, EAC3, DTS, FLAC, WAV, MP3, OPUS или OGG.

GUI не просит выбирать source-формат вручную. Backend проверяет каждый файл и пропускает несовместимые случаи с записью причины в лог. Same-container copy разрешён, если меняется состав потоков, например MP4 без аудио.

Как разворачивать. Отдельная строка кнопок в этом же разделе: **как снято**, **90 по часовой**, **90 против часовой**, **перевернуть**, **зеркало по горизонтали**, **зеркало по вертикали**.

Это лечит знакомую боль: камеру повернули набок ради вертикального кадра, а она записала обычный горизонтальный файл и ничего об этом не сказала — и дальше каждый клип приходится разворачивать в монтажке руками. Здесь разворот записывается в контейнер, и NLE открывает клип сразу правильно.

Пиксели при этом не трогаются вовсе. Измерено на дубле 960×540: файл был 6 085 307 байт, стал 6 085 304 — разница только в заголовке, хэши кадров совпадают до единого, а декодер выдаёт 540×960.

Границы, проверенные, а не предположенные:

- хранят разворот **MP4, MOV и MKV**. **MXF — нет**: он принимает команду и молча пишет неразвёрнутый файл, поэтому раздел отказывает до запуска;
- предлагается для камерных **H.264 и HEVC**; ProRes и DNxHR пропускаются с объяснением;
- это указание тому, кто показывает файл. Resolve, Premiere и mrv его читают, простой просмотрщик может не прочитать.

Обрезка

Отрезает начало, конец или середину камерного файла **без перекодирования**: потоки копируются, поколение не теряется, метаданные и таймкод переезжают в результат.

Что выбрать. Три кнопки, и все три — про кусок между точками.

Оставить — остаётся то, что между IN и OUT, остальное уходит. Какой именно край уйдёт, уже сказано полями, и повторять это кнопкой незачем: один IN — уходит начало (хлопушка, подход, фальстарт), один OUT — уходит конец (отход и остановка камеры), обе точки — уходят оба края.

Вырезать — наоборот: кусок между точками исчезает, а начало и конец склеиваются в один файл. Оговорка, телефонный звонок, прошедший через кадр ассистент. Перекодирования нет и здесь: два куска копируются потоками и склеиваются пакет к пакету.

Разделить надвое — точка одна, ничего не выбрасывается, получаются два файла.

У выреза одно правило работает наоборот к остальным резам. Везде начало куска ложится на ближайший ключевой кадр, даже если он раньше запрошенного, — лишние кадры ничего не стоят. Здесь стоят: часть того, что просили убрать, осталась бы в результате. Поэтому хвост начинается с **первого ключевого кадра после OUT**, и журнал говорит, насколько больше запрошенного ушло. Если ключевой кадр стоит ровно в точке — не ушло ничего лишнего.

Звук на стыке режется по сэмплу: РСМ переписывается тем же форматом, и куску задаётся его собственная длина, иначе звук обрывается вместе с последним кадром. Измерено на дубле 25 к/с: без этого стык уезжал на 72 мс, затем на 16 мс; сейчас результат — ровно 1 200 000 сэмплов на 25,000 с.

Как ставятся точки. Двумя способами, и оба точные.

Плеером. Кнопка **Открыть в плеере** показывает выбранный дубль в mrv. Там пробел играет и останавливает, **,** и **.** шагают ровно на кадр назад и вперёд, **b** включает воспроизведение задом наперёд, **l** ставит точку A, второе нажатие — точку B, и кусок между ними начинает играть по кругу: рез слышно до того, как он сделан. Дальше **Взять A и B** переносит обе точки в поля.

Если удобнее держать руки на панели — **Отметить IN здесь** и **Отметить OUT здесь** берут позицию, на которой стоит плеер, пишут её в поле и переносят туда же точку петли, так что кусок продолжает крутиться.

Плеер живёт отдельной программой: его можно держать на втором мониторе, а если он закроется — панель останется на месте. Ставится он один раз: **install\Install-Portable-**

`mpv.cmd`.

Клавиатурой. Таймкодом: `00:01:28,400`, или просто секундами. Способ, которому не нужно ничего, кроме цифры, — и он остаётся, даже если плеера нет.

Что раздел обещает. Начало куска ложится на ближайший ключевой кадр перед указанной точкой — раньше него копируемый поток нечем начать, и журнал говорит, куда оно попало на самом деле. Конец считается кадрами по точной дроби частоты, поэтому попадает ровно в запрошенный кадр. Частота берётся у файла как есть: `ffprobe` отдаёт точную дробь, и ничего не домысливается — камера снимает и 23,976, и честные 24,000.

Когда раздел откажет — до запуска, а не после. MP4 не примет ProRes, MXF не примет сжатый звук, запрос несуществующей звуковой дорожки отклоняется, а камерный RAW в MXF (Sony X-OCN, ARRIRAW) FFmpeg не читает вовсе — о каждом случае сказано словами и названа рабочая альтернатива.

То же самое из командной строки: `Scripts\ff-trim-start.cmd`, `ff-trim-end.cmd`, `ff-trim-both.cmd`, `ff-trim-middle.cmd`. Арифметика общая с панелью: файл, вырезанный обёрткой, и файл из панели совпали побайтово.

Подробности, таблицы совместимости и измерения — в `Docs\TRIM_MATRIX_RU.md` и `Docs\MEASUREMENTS_RU.md`.

FPS / Частота

Исходный FPS читается из видеопотока. Пользователь выбирает:

- `Varispeed` или `Conform`;
- целевую частоту из полного ряда: `16`, `18`, `23.976`, `24`, `25`, `29.97`, `30`, `48`, `50`, `59.94`, `60`. Дробные считаются точными дробями; исходная частота может быть любой, вплоть до скоростной съёмки (проверено на 8000 fps);
- профиль вывода: H.264 MP4, H.265 MP4, ProRes MOV/MXF, DNxHR MOV/MXF.
- для ProRes/DNxHR в Apple MOV: PCM 16/24/32-float и сохранение рабочей частоты до 176.4/192 кГц;
- для ProRes/DNxHR в MXF: PCM 16/24 и финальные 48 кГц; `2x/4x` остаётся внутренней частотой обработки;
- те же профили, разрядности и `x1/x2/x4` доступны в CLI FPS-мастере; итоговая команда строится общим контрактом GUI/CLI.

Цвет / LUT

Раздел для LUT и HDR-to-SDR:

- LUT picker/cache из `LUTs\`;
- `LUT x264`, `LUT HEVC Main10`, `LUT ProRes 422`;
- `Pre-expose + LUT`;
- `Pregamma + LUT`;
- `HDR to SDR Hable`;
- targets: x264, x265, SVT-AV1, ProRes 422, ProRes LT.

Для pregamma/pre-expose есть 18% gray preview. Он показывает направление изменения серого до LUT.

Hardware Stack

Проект умеет выбирать:

- CUDA / NVENC;
- Intel QuickSync / QSV;
- AMD AMF / D3D11VA;
- dav1d для AV1 decode;
- SVT-AV1 для AV1 encode;
- CPU fallback.

Доступность зависит от FFmpeg build, драйверов и железа. Hardware-профили сверяются с cache из `Hardware Capabilities`.

Терминал, Прогресс И Отчёты

Правая область содержит:

- статус операции;
- общий progress bar;
- терминальный лог;
- `Команда` для dry-run;
- `Logs`;
- `Report`;

- `CONFIG`;
- разворот терминала;
- ручную командную строку с Shell, CWD, историей и пинами.

Каждая медиа-операция пишет `batch_report.md` и `batch_report.json` в `report\...`.
Аудио пишет дополнительный `audio_report.md/json`.

Правила Безопасной Работы

- Сначала `Команда`, потом реальный запуск, если операция новая.
- Для больших папок сначала `Тест: первый файл`.
- Не включай `Перезаписывать`, если не уверен в OUT.
- Для hardware backend сначала `Hardware Capabilities`.
- Для LUT сначала проверь выбранный `.cube`.
- Для YouTube batch сначала проверь `Download\urls.txt`.
- Для cleanup внимательно смотри выбранный Source/OUT.
- Не запускай две операции в одном GUI одновременно.

Проверка После Изменений

```
runtime\python.exe -m py_compile system_core\ui_nicegui\app.py
system_core\services\media_service.py Scripts\Common\script_runner.py
runtime\python.exe system_core\ui_nicegui\app.py --smoke
```

Для проверки зеркального OUT можно использовать тест из

`Docs\docs\MEDIA_SMOKE_TEST_CHECKLIST_RU.md`.